

Multi-Agent Systems: When One Detective Isn't Enough

Chapter 6, objective 1.4. The four properties, the three topologies, the caps to quote, and the coordinator-over-the-coordinator trap.

The agency, not the lone detective. Some cases outgrow one detective. The customs specialist, the carrier negotiator and the customer handler each need different tools, different briefs, different clearances. An agency has one partner who assigns cases and reads reports, and a strict rule: field notes stay in the field office; only the report crosses the partner's desk. That rule is context isolation, and it is why agencies scale where lone detectives drown.

Multi-agent means agents delegating to agents. In the Agent SDK, subagents are separate agent instances the orchestrator spawns, each running in its own conversation. Four properties carry the exam weight.

Property	Mechanism	Why the exam cares
Context isolation	A subagent explores in its own window; only its final message returns to the parent	A research subagent can read 40 documents without flooding the orchestrator. Domain 2's token economics, solved architecturally
Parallelism	Independent subagents run concurrently	Three reviews finish in the slowest one's time. E4's logic at agent scale
Specialised prompts	Each subagent gets its own system prompt and expertise	The customs agent's SOP knowledge would be noise in the negotiator's context
Tool restriction	Each subagent gets only its own tools; an omitted tool does not exist for it	The customer-facing agent physically cannot call process_refund. Configuration beats instruction, again

The delegation brief is everything. A subagent starts **fresh**, with no parent history, so the orchestrator's prompt to it must carry every path, id and decision it needs. Structured, complete handoffs between agent steps were your other 100 percent row at Foundations. At Professional they are load-bearing.

Everyday example: a hospital shift handover. The night doctor cannot say the patient in bed four, you know the one. The day doctor was not there. The handover sheet must carry the name, the history, the allergies and what was decided at 3 a.m., or the day shift starts from zero.

Choosing a topology

Topology	Shape	Reach for it when	It hurts when
Single agent	one loop, all tools	tools are a handful, one expertise	context bloats; one prompt serves three masters
Orchestrator plus subagents	partner assigns, field offices work, only reports travel	distinct expertises, parallelisable, isolation is valuable	subtasks so entangled every report needs every other's field notes; the isolation you bought becomes re-briefing overhead
Peer handoff	agent A completes, passes the baton to B	sequential phases with a clean break: triage shift then resolution shift	the baton is informal; an under-specified handoff drops context and authorization state

Everyday picture of the three: a general practitioner working alone (single agent); a GP who refers to specialists and reads their letters (orchestrator plus subagents); a hospital day shift handing over to the night shift (peer handoff).

Drift: guardrails around subagent growth. Common teaching: subagents can nest and parallelise, with no numbers attached. The brief's reading of current SDK docs names hard caps you set: spawn depth (default 3 layers, 1 means no nesting), concurrency (default 20 at once), and a per-query spend cap (max_budget_usd) that refuses new spawns, stops background subagents and ends the query at the limit. That is Law 3 at fleet scale. The spawning tool now appears as Agent in tool_use blocks; older versions said Task. These are Agent SDK facts from the brief; the Messages API reference I loaded does not cover them, so quote them as the brief gives them.

Trap: a coordinator over the coordinator. Adding a subagent is often correct. Adding a second orchestrator to manage the orchestrator almost never is. When two options both delegate, prefer the one with fewer layers and the same isolation. And remember the places where the agent or subagent genuinely is the answer: unpredictable subtasks, isolation, parallel expertise.

Domain 3 connection

Tool restriction is the lab's E3 gate and E2 per-route belt seen from the agent's side. The Wrong Tower flaw where permissions lived in the system prompt is the same lesson: never process refunds as an instruction is persuasion; omitting the tool from the subagent's list is physics. Context isolation is why the One Ticket window stayed at about 5,800 tokens: the retrieval happened somewhere that was not the drafting model's context.

The one thing to remember. Isolation, parallelism, specialised prompts, tool restriction. A subagent starts fresh, so the brief carries everything. Caps, not committees: depth, concurrency and spend. A second orchestrator is almost always the wrong answer.